

Trabalho 4: Sistema de Autocomplete de Jogos com Trie

Relatório de Implementação e Análise Computacional

Luis Eduardo Weigert Weiss
Murilo Granemann de Souza

7 de junho de 2026

1 Representação da Trie

A estrutura fundamental para o sistema de autocomplete foi baseada em uma árvore de prefixos (Trie) cuja representação interna requer a definição precisa de seu alfabeto e o mapeamento adequado de suas chaves.

Tamanho do Alfabeto e Mapeamento: O alfabeto adotado consiste em **36 caracteres**, contemplando as 26 letras do alfabeto inglês e os 10 dígitos numéricos (0 a 9). Esse dimensionamento foi escolhido para acomodar perfeitamente os títulos dos jogos disponíveis no banco de dados. Internamente, os caracteres são convertidos em índices da matriz de filhos `children[36]` da seguinte maneira:

- Para as letras ('a' até 'z'), subtrai-se o valor ASCII de 'a', gerando os índices de 0 a 25.
- Para os números ('0' até '9'), subtrai-se o valor ASCII de '0' e soma-se 26, originando os índices de 26 a 35.

Normalização de Títulos e Prefixos: Para converter os textos brutos nas chaves de busca padronizadas utilizadas na Trie, o método `toSearchKey` realiza três etapas críticas de normalização: ignora qualquer espaço em branco (' '); converte letras maiúsculas para minúsculas mediante adição direta do desvio ASCII ('a' - 'A'); e descarta quaisquer outros caracteres não alfanuméricos, anexando apenas os válidos à string de retorno. Dessa forma, as buscas se tornam *case-insensitive* e imunes ao espaçamento equivocado.

2 Funcionamento dos Métodos

Em alto nível, as operações vitais da Trie estão encapsuladas em três métodos centrais:

- **insert:** Extrai a chave de busca normalizada a partir do título do objeto `Game` referenciado e itera caractere a caractere, traduzindo-os em índices de 0 a 35. Durante a descida na árvore, instanciam-se novos objetos `TrieNode` nos caminhos ainda inexistentes. Ao final do percurso, o último nó recebe `isEndOfTitle = true` e guarda o ponteiro para o `Game` associado.

- **contains:** De maneira análoga à inserção, converte a string fornecida e tenta navegar na Trie descendo pelos nós segundo o mapeamento. Caso a execução depare-se com um ponteiro `nullptr` na matriz de filhos antes de esgotar a string, ou caso atinja o último nó e este não for um fim válido de título, o método retorna `false`. Apenas o atingimento de um nó marcado como `isEndOfTitle` atesta o sucesso.
- **autocomplete:** Primeiramente, normaliza o prefixo solicitado e traça um caminho exato na estrutura até alcançar o *nó-raiz da subárvore* que compreende todas as continuações possíveis desse prefixo. Caso o prefixo não exista, um vetor vazio é devolvido. Caso o nó seja alcançado, aciona-se um método auxiliar recursivo de travessia em profundidade (*Depth-First Search*), que coleta todos os ponteiros de jogos alocados nessa subárvore. Posteriormente, os jogos reunidos sofrem ordenação (implementada via *Insertion Sort*) que prioriza a popularidade de forma decrescente e aplica desempate através da comparação lexicográfica crescente da chave de busca de seus títulos. Finalmente, o conjunto de dados é fatiado para retornar, no máximo, os top- k elementos desejados.

3 Análise de Custo

O custo computacional das operações na Trie deve ser interpretado em função do comprimento dos termos consultados e do perfil morfológico da árvore.

- **Custo do insert:** $\mathcal{O}(L)$ — A complexidade temporal depende unicamente de L , o tamanho do título após sua normalização para a chave de busca. Como a alocação e indexação nos arrays custam $\mathcal{O}(1)$, a criação do percurso domina o custo.
- **Custo do contains:** $\mathcal{O}(L)$ — Semelhante à inserção, a navegação para um teste de pertencimento exato apenas atravessa as ligações até L de profundidade máxima antes de fornecer o veredicto.
- **Custo do autocomplete:** $\mathcal{O}(p + s + r^2)$ — Para atender ao comando, percorrem-se três fases: o alcance do prefixo custa $\mathcal{O}(p)$; em seguida, a DFS caminha por $\mathcal{O}(s)$ nós presentes na subárvore para colher os alvos; por fim, a ordenação de todos os r achados via *Insertion Sort* apresenta um custo em tempo quadrático de $\mathcal{O}(r^2)$ no pior caso.

4 Comparação e Uso de Memória

Vantagem Assintótica na Busca: Ao compararmos nossa estrutura à abordagem ingênua (na qual todos os N jogos estariam salvos num simples vetor sequencial), atesta-se que o ganho temporal da Trie é imenso para sistemas grandes. Em um array, para cada autocompletar, um varrimento de custo $\mathcal{O}(N \cdot p)$ é engatilhado apenas para identificar compatibilidades, seguido de processamentos pesados de ordenação. Na Trie, descarta-se quase integralmente o catálogo, pois a exploração avança imediatamente para os caminhos que satisfazem a restrição espacial do prefixo, poupando a varredura linear por incompatíveis.

Impactos no Custo de Memória: Como *trade-off* pelo seu desempenho superior no tempo computacional, o uso de árvores de prefixo exige dispêndios massivos de memória, decorrentes principalmente do dimensionamento estático da matriz contida no nó. Com nosso alfabeto tamanho 36, tem-se a carga impositiva de alocar 36 blocos de memória por cada **TrieNode** existente na estrutura, que na grande maioria permanecem inativos e preenchidos apenas com **nullptr**. O agravante do espaço é um sacrifício aceitável; entretanto, o compartilhamento eficiente de prefixos — como entre “Halo”, “Hades” e “Half Life”, todos absorvidos sob o caminho basal inicial ‘h’ → ‘a’ — condensa amplamente os desperdícios que um mapeamento distinto produziria. Reduzir ou ampliar a gama de símbolos disponíveis impactaria diretamente as ramificações e, conseqüentemente, esse preenchimento disperso do espaço operacional.